



For this assignment, write a C++ program that keeps track of some Birds, but this time, use a set of two classes. To the user of the program, this version will seem just like the last version (from [assignment 1](#)), but it will work very differently. Modern programs should be portable, are written by teams, and need to be updated from time to time, so even though you will code this assignment by yourself, you will be implementing two key object-oriented programming concepts: **encapsulation** and **data hiding**. Encapsulation is the reason why we need two classes. I'm going to call the first class Bird, but you could use some other name, like BirdInfo. It will be very similar to the struct you created from assignment 1 but will have encapsulated

member functions. The other class you will create is for a collection of Birds. We use collections so often for things that we have names for them. For example, a **deck** of cards or a **flock** of birds. I'm going to use the name BirdList for a collection of Bird objects. Another name might be SightingList. You could use Birds to save space, but that only differs by 1 letter. Start the name of the class with a capital letter. See below for examples of what the classes might look like. Continue to use the same format from assignment 1 for loading from a file, presenting a menu driven loop, error checking, etc. Load the input file at the start of the program, close it and only deal with data in memory, and then write the data back to the output file at program termination. This assignment is split into 3 versions, A B and C, just like assignment 1. See below for details. Click this link, [assignment 1](#), to view the Google Doc version. You can use assignment 1 as the starting point for this one.

Requirements:

The BirdList class must encapsulate the array of Bird class objects, and the array must be private. **You may not have a separate array of Bird objects defined anywhere else, such as main() or as a global.** All functions that manipulate data for the BirdList class (loading from the file, searching for a Bird, etc.) must be encapsulated inside of the BirdList class. In other words, they must be member functions, commonly referred to as methods. All member variables (properties) of the Bird class must be encapsulated and private inside of the Bird class. These include name, location, weight, sightings, rarity, and description. All functions that manipulate data for the Bird class must be encapsulated inside of the Bird class, commonly referred to as getters and setters.

Do not use <string>, <vector>, or any other stl libraries. You must continue to use only c-strings (null-terminated char arrays) to store words and only use regular arrays rather than any container classes such as <vector>. You must use functions and have multiple code files and header files. You must follow the coding style listed in the [C++ style guidelines](#).

Tasks:

- Complete the zylabs from chapter 17 and 18. [17.11, Artwork Label](#) and [18.4, Car Value](#). There are examples and a video in section [17.10, Nested Classes](#), that may help with the concepts in this assignment. It has a Playlist class, which is a collection of SongType objects. For chapters 17 and 18, please complete the zylabs with a score of 100%, 70% of the challenge activities, and 80% of the participation activities.
- Open the [Algorithm Design Document](#) or the [UML based Algorithm Design Document](#), make a copy, and follow the steps to create your algorithm. Complete the algorithm design document to receive a grade (your assignment will not be graded until you provide a design document).
- If you are tired of creating pseudocode and would like a graphical option, you can use this alternative [UML based Algorithm Design Document](#). It has an example of a UML graph instead of pseudocode.
- Load the data from the database file in sorted order by name. You may not use a user-defined sorting function or sort function from a library - this means that each Bird must be inserted into the array in sorted order.

After completing this assignment, you will be able to:

- Write code using classes and objects.
- Implement the default constructor for a user-defined class.
- Organize source code in multiple header and implementation files (.h and .cpp files).
- Implement and use accessors and mutators for a class.
- Use C-strings to represent character strings.

- Load the data when the program is started and write the data back to the file at program termination. Only load from the file at the start of the program. Only write to the file at program termination.
- After the file is loaded, the program will enter a user-controlled loop. Inside of the loop, the program will ask the user what they want to do. See assignment 1 for an output example.
- The rest of the tasks are split into 3 different versions based on the type of grade you want, just like assignment 1. There are 3 versions for this assignment, and each version has the minimum requirements for a particular grade – A, B or C. The B version builds on the requirements of the C version, and the A version builds on the requirements of the B version. Here are the requirements for each version:

Version C:

- Load the Birds from the file in sorted order by name.
- List all the Birds, one per line, using line numbers.
- Add a new Bird to the database in sorted order.
- Write the data out to a file and quit.

Version B:

- Load the Birds from the file in sorted order by name.
- List all the Birds, one per line, using line numbers.
- Add a new Bird to the database in sorted order.
- Remove a Bird by index (**new for version B**).
- Write the data out to a file and quit.

Version A:

- Load the Birds from the file in sorted order by name.
- List all the Birds, one per line, using line numbers.
- Add a new Bird to the database in sorted order.
- Remove a Bird by index.
- Search for a Bird by name (**new for version A**).
- List all Birds by Location (**new for version A**).
- Write the data out to a file and quit.

```
class Bird {
    char name[STR_SIZE];
    char location[STR_SIZE];
    double weight;
    int sightings;
    int rarity;
    char description[STR_SIZE];

public:
    Bird(); // default constructor.
    // Getters
    const char * getName() const;
    const char * getLocation() const;
    double getWeight() const;
    int getSightings() const;
    int getRarity() const;
    const char * getDescription() const;
    // Setters
    void setName(const char *);
    void setLocation(const char *);
    void setWeight(double);
    void setSightings(int);
    void setRarity(int);
    void setDescription(const char *);
};
```

```
class BirdList {
private:
    int count;
    char fileName[STR_SIZE];
    ifstream inFile;
    Bird list[ARR_SIZE];
    // private method.
    bool insert(Bird toAdd);

public:
    BirdList(); // Default constructor.
    ~BirdList(); // destructor.
    int loadBirds();
    void printBirds() const;
    bool search(const char *) const;
    void printByLocation() const;
    bool addABird();
    bool removeABird();
    void writeBirds() const;
};
```

NOTE: There are different error checking requirements when adding a new Bird or removing a Bird for each one of the grade versions above, just like assignment 1:

- C version
 - Check to make sure the weight is greater than 0.05.
 - Check to make sure the sightings and rarity values are positive.
- B version
 - Check to make sure the weight is greater than 0.05 and less than or equal to 1600.0.
 - Check to make sure the sightings value is positive and less than or equal to 100.
 - Check for proper array index to remove (not out of bounds – too low or too high).
- A version – All the error checks for the B version and check for input failure (don't allow the program to crash or go into an infinite loop when letters are typed for input to number variables).

Make sure that there is a newline (return) at the end of every line of the database file, including the last line. See the assignment 1 document for handling the newline at the end of the file. There is an example data text file on D2L, along with this document. The first 10 birds are also listed out in assignment 1.

The sample run / sample output is the same as the first assignment, so it has been removed from this version. Open [the first assignment document](#) to see the sample output.

Academic Integrity



You may NOT, under any circumstances, begin a programming assignment by looking for completed code on StackOverflow or Chegg or any such website, which you can't claim as your own. Please check out the [Student Code of Conduct at PCC](#).

If you start with code from another source and just change the variable names or other content to make it look original, you will receive a zero on the assignment. I may ask you to explain your assignment verbally. If you cannot satisfactorily explain what your code does and why you wrote it in a particular way, then you should not expect to receive credit. Do not share your code with any other students and do not post it on a public website.

Programming Requirements

- Now that we have two classes for this assignment, we need a few more files. Just like assignment 1, you must have a main.cpp file, Bird.h and Bird.cpp files, but we also need header and implementation files for BirdList: BirdList.h and BirdList.cpp. Place the Bird class definition in the Bird.h file and the member function definitions in the Bird.cpp file. Place the BirdList class definition in the BirdList.h file and the member function definitions in the BirdList.cpp file. You may change the name of the code files if you choose different name for your classes, such as Birds.h and Birds.cpp. All member function definitions must be part of the class implementation file for both classes. You may also have `tools.h` and `tools.cpp` or `functions.h` and `functions.cpp`, but these files should only contain code for stand-alone functions (such as `welcome()`) which are not part of a class. You can also add a `main.h` file if you want. So here is the list of files that may be part of your assignment:

Bird class files:	Bird.h	Bird.cpp
BirdList class files:	BirdList.h	BirdList.cpp
Main function files:	main.cpp	main.h (optional)
Tools or functions files:	tools.h or functions.h	tools.cpp or functions.cpp

- Just like assignment 1, Don't use the string library or any of the STL containers such as vectors.
- Encapsulate your array of Bird objects in the BirdList class. Do not use any global variables.
- Just like assignment 1, you must create several functions for the Bird array, but they must be part of the BirdList class, and therefore, in the BirdList header and implementation files. There are at least four member functions that you must create for the C version, including: `loadBirds()`, `writeBirds()`, `addABird()`, and `printBirds()`. For the B version, you must add `remove()`, and for the A version, `printByLocation()` and `search()`. Since these functions are member functions, they will have the class name and the scope resolution operator in the function header. Notice that because these functions use the member properties, they don't have arguments:
`bool BirdList::addABird();` //return type (bool, void, etc.) is up to you.
If you add other stand-alone functions, such as `welcome()` place them in a tools or functions header and implementation file. You may change the name of these functions, such as `removeByIndex()` instead of just `remove()`.
- Just like assignment 1, make sure you check for index out of bounds before adding new data to the array.

- Notice that the `insert()` function is in the private section of the `BirdList` class. Only the `addABird()` and `loadBirds()` functions will be calling it so it should be private (not available to client code).
- Check to make sure that the input file is open before reading from it, using `ifstream.is_open()`. Ask the user to try a different file name if the file does not open or have them type “quit” or “exit” to quit the program at that point (if they don’t have a collection file).
- Make sure that you check for negative numbers and bad data when reading number data from the user.
- For version C, make sure the number values are positive (greater than zero).
- For version B, check the values for version C, but also make sure the values are less than the values spelled out in [Tasks - B version](#). For removal in version B, make sure that the index is not out of bounds.
- For version A, do all the error checking for version B but also make sure your program doesn’t crash or go into an infinite loop for input failure.
- Use the file guards for your header files: `#ifndef`, `#define` and `#endif`. These are the preprocessor directives that should be placed around the contents of a header file.
- The `BirdList` class will make use of the `Bird` class, so use `#include "Bird.h"` in the `BirdList.h` file. The main function will make use of the `BirdList` class, so use `#include "BirdList.h"` in `main.cpp` or `main.h` (if you have a `main.h` file). That way, the header file for `Bird` will be included in both the `BirdList` implementation and in the main function file.
- You must represent a single `Bird` using a `Bird` class object. Represent all the `Birds` in the list using the `BirdList` class. To be precise, you must have an array of `Bird` objects inside of a `BirdList` class object, and you may not create an array outside of `BirdList`, such as in `main`.

Programming Strategies



- Notice that there is a prototype for the default constructor in both classes. Default constructors are invoked when a new object is created. Anything about an object that should be set up automatically should happen in the default constructor. For example, when a new `BirdList` object is created, the array is empty, so the count property should be set to zero.
- Notice that all the getter member function prototypes and other functions that don’t modify the properties, only read them, have the word `const` at the end. This is to allow the compiler to help you avoid mistakes by changing a property. Place `const` at the end of the headers as well.
- Notice that `search()` has a `bool` return type and takes a c-string argument. This is so the function can return success or failure and allow the main function to ask for the search name. If you prefer to handle these details inside of the function, you can change the return type to `void` and eliminate the argument.
- Notice that none of the other member functions (methods) of the `BirdList` class have arguments. That’s because all the arguments that were used in assignment 1 have been encapsulated. Let’s compare one of the functions for assignment 1 to the prototypes in the `BirdList` class – `loadBirds()` for example:
 - Assignment 1: `int loadBirds(Bird Birds[], ifstream & inFile);`
 - Assignment 2: `int loadBirds();`
 Why doesn’t the version from assignment 2 need the arguments? The answer is because all the arguments – the array of `Birds`, the input file, and/or the file name – have all been encapsulated and are members of the class. So, they can be directly accessed by the `loadBirds()` function.
- You can encapsulate as many properties as you need for the classes, including the file name, input file, and possibly the output file for the `BirdList` class. I’ve included the file name and input file as part of the `BirdList` class in the textbox above. If I forgot some properties or methods that should be part of either class, go ahead and add them and let me know. If you want to add something about individual `Birds`, make it part of

the Bird class. If you want to add something about the collection, add it to the BirdList class. You can ask for the file name and open the file in the constructor or in the `loadBirds()` function. Don't forget to close the file at the appropriate time. Use the same name or ask the user for an output file name for an ofstream file for `writeBirds()`.

- You may assume that the input file is formatted correctly, so no error checking is necessary when reading from the file. See the assignment 1 document for data verification strategies for data from the keyboard.
- You can use the example database file, or you can create your own input file using your favorite text editor on the Linux system: vim, emacs, nano, etc. If you use a Windows or Mac program like Notepad++ or TextEdit, make sure that the last character in the file is a newline (return character), and make sure you save the input file in text file format with Unix line termination, rather than rich text, Word, or some other format. Use the <fstream> library to load the file. You must use C++-style input and output objects and manipulators; do not use any C-style functions for input and/or output, such as `fprintf`, `fscanf`, etc.
- When doing list output, you may use `setw()` and `left/right` from the <iomanip> library to align the data into columns, but it is not required.
- Do not assume that the input file contains a certain number of Birds. Read from the file using an EOF-controlled loop (loop to the end-of-file marker. See the assignment 1 document for more details).
- Do not use extraction (>>) with `cin` or `ifstream` to read data into any c-strings, because this is a potential security issue called buffer overflow, and extraction also stops at whitespace. Use `cin.getline()` or `infile.getline()` instead. For more information, check out the cplusplus.com page.

List of Tasks to Complete:

- Complete zyBooks [zylab 17.11, Artwork Label](#) and [zylab 18.4, Car Value](#)
- Open the [Algorithm Design Document](#) or [the UML based design document](#), make a copy, and follow the steps to create your algorithm.
- **Be sure to include your screenshot of the zyBooks completion in the algorithm document. The screenshot must be present for your assignment to be graded.**
- Create your source code files, build and test your program.
- Code may be written using any development environment but must use Standard C++.
- PSU-bound students are strongly encouraged to do all development on the PCC Linux server.
- Please open and compare your work with the [grading rubric](#) before submitting.
- Remember to follow all [C++ style guidelines](#).
- After you complete the code and the algorithm document, upload the files along with your text file to the D2L assignment area. You may compress the files together and upload a zipped file if you prefer.
- You may express your algorithm as pseudocode or a flowchart. Please note that your pseudocode must follow the syntax requirements shown in the document - **you may not use C++ syntax as a substitute for pseudocode.**

Additional Support

- Post a question for the instructor in the Ask Questions! area of the Course Lobby.
- Set up a Zoom session with the instructor.
- Come to class in person or on Zoom on M/W from 10:00 - 12:50.
- Go to the Discord server and post a question there.